

🔧 手搓教程：从零搭建「安克耳机语音识别 (WeNet U2++)」

适合人群：**完全小白**。你只要会用鼠标、会打字、会复制粘贴就能跟下来。

工具：VSCode 或 PyCharm 任选其一（本教程两个都给步骤）。系统：Windows。

核心理念：分阶段做，每做完一步立刻运行验证（看到 ☒ 再进行下一步）。

千万不要一口气把所有文件都建好才运行——那样出错很难定位。

桌面上 `anker-asr-wenet/` 里我已经放了全部"标准答案"文件。本教程带你**理解并亲手重建**它们。每个文件你可以：

- 方式A（推荐学习）：看懂作用后自己对着标准答案敲一遍；
- 方式B（赶时间）：直接用现有文件，按本教程的顺序运行、验证。

目录

- [第 0 部分：准备工具（装一次，终身受用）](#)
- [第 1 部分：建项目 + 配 Python 环境](#)
- [第 2 部分：装依赖](#)
- [第 3 部分：先理解模型（跑教学版 U2++）](#)
- [第 4 部分：生成语音数据集](#)
- [第 5 部分：准备 WeNet 数据格式](#)
- [第 6 部分：计算 CMVN](#)
- [第 7 部分：安装 WeNet 训练代码](#)
- [第 8 部分：写训练配置 + 开始训练](#)
- [第 9 部分：解码评测（算 CER）](#)
- [第 10 部分：实时识别 Demo](#)
- [第 11 部分：一键脚本（收尾）](#)
- [附录：报错急救表](#)

第 0 部分：准备工具

这一部分装 4 样东西。装完每一样都用命令验证一下。

0.1 装 Python (3.9 ~ 3.11)

1. 浏览器打开 <https://www.python.org/downloads/>，下载 **Python 3.10**（别用最新的 3.13，太新有些库还没适配）。
2. 双击安装，**第一个界面一定要勾选 ☒ 「Add Python to PATH」**，再点 Install Now。
3. **验证**：按 `win` 键，搜 "PowerShell"，打开它，输入：

```
python --version
```

看到 `Python 3.10.x` 就成功。

0.2 装 VSCode 或 PyCharm（二选一）

- **VSCode**: <https://code.visualstudio.com/> 下载安装。装完打开，点左侧方块图标（Extensions），搜 **Python**，安装微软官方那个 Python 扩展。
- **PyCharm**: <https://www.jetbrains.com/pycharm/> 下载 **Community（社区免费版）** 即可。

两个都装也行，本教程后面"建项目/建虚拟环境/运行"两种都给步骤。

0.3 装 ffmpeg（把合成的音频转成标准 wav 要用）

1. 在 PowerShell 里执行：

```
winget install --id=Gyan.FFMpeg -e
```

（如果提示找不到 winget，去微软商店搜"应用安装程序"装一下，或百度"ffmpeg windows 安装"手动装。）

2. **关键**：装完要把 **PowerShell 关掉重开一个**（让环境变量生效）。

3. **验证**：新开的 PowerShell 里输入：

```
ffmpeg -version
```

能打印一堆版本信息就成功。

0.4 装 Git（下载 WeNet 代码要用）

1. <https://git-scm.com/download/win> 下载安装，一路下一步即可。

2. **验证**：新开 PowerShell：

```
git --version
```

✅ **检查点**：`python --version`、`ffmpeg -version`、`git --version` 三条命令都能正常打印版本号 → 进入下一部分。

第 1 部分：建项目 + 配 Python 环境

1.1 建项目文件夹

如果你想完全自己搓，新建一个空文件夹；如果想对照我给的标准答案，直接用桌面那个 `anker-asr-wenet`。这里以自己新建为例。

在桌面新建一个文件夹，命名 `my-anker-asr`（名字随意，别用中文和空格）。

1.2 用 IDE 打开这个文件夹

VSCode:

- 菜单 `File → Open Folder...`, 选中 `my-anker-asr`, 打开。

PyCharm:

- 启动页点 `Open`, 选中 `my-anker-asr`, 打开。

1.3 创建"虚拟环境" (venv) —— 非常重要

虚拟环境 = 给这个项目单独开一个干净的 Python "小房间", 装的库不污染系统。

方式一: 用 PowerShell 手动建 (VSCode/PyCharm 通用, 最稳)

在 IDE 里打开终端:

- VSCode: 菜单 `Terminal → New Terminal`。
- PyCharm: 左下角点 `Terminal` 标签。

确认终端开头路径是你的项目文件夹, 然后执行:

```
python -m venv .venv
```

执行完项目里会多出一个 `.venv` 文件夹。接着激活它:

```
.\.venv\Scripts\Activate.ps1
```

⚠ 如果报红字 `无法加载...因为在此系统上禁止运行脚本`, 执行下面这条解除限制, 再重新激活:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

激活成功的标志: 终端每行最前面出现 `(.venv)`。

方式二: PyCharm 图形界面建

- `File → Settings → Project → Python Interpreter` → 右上角齿轮 `Add` → 选 `Virtualenv Environment` → `New` → 确定。PyCharm 会自动建好并选用。

1.4 让 IDE 用上这个虚拟环境

VSCode: 按 `Ctrl + Shift + P`, 输入 `Python: select Interpreter`, 回车, 选列表里带 `.venv` 字样的那个。

PyCharm: 如果用方式一手动建的, 去 `Settings → Project → Python Interpreter` → `Add` → `Existing` → 选 `.venv\Scripts\python.exe`。

✅ **检查点:** 终端前面有 `(.venv)`, 且 IDE 右下角/解释器设置里显示的是 `.venv` 里的 Python → 成功。

第 2 部分：装依赖

2.1 创建 requirements.txt

在项目根目录新建文件 `requirements.txt` (VSCode: 右键项目 → New File; PyCharm: 右键项目 → New → File)。

内容直接照搬桌面标准答案里的 `requirements.txt` (里面列了 torch、torchaudio、edge-tts、soundfile、librosa、sounddevice、tqdm、pyyaml 等)。

这个文件的作用：把项目要用的所有 Python 库列在一起，一条命令批量安装。

2.2 安装

确认终端有 `(.venv)`，执行：

```
pip install -r requirements.txt
```

这一步会下载不少东西 (torch 比较大)，耐心等几分钟。

注：wenet 这一行可能装的是推理库、不含训练入口，没关系，第 7 部分会专门处理训练代码。

✓ **检查点：**执行 `python -c "import torch, torchaudio, edge_tts, soundfile; print('deps ok')"`，打印 `deps ok` → 依赖装好了。

第 3 部分：先理解模型（跑教学版 U2++）

为什么先做这个？因为它**不依赖 WeNet**、**不依赖数据**，能让你立刻“看见”U2++ 内部长啥样，对写报告/答辩帮助极大。先尝甜头，建立信心。

3.1 创建 model/u2pp_conformer_min.py

新建文件夹 `model`，里面建文件 `u2pp_conformer_min.py`，内容用桌面标准答案。

你要理解的 4 个部分（答辩会问）：

- `ConformerBlock`：1/2前馈 → 多头自注意力 → 卷积模块 → 1/2前馈（这就是 Conformer 的“三明治”结构）。
- `ctc_fc + F.ctc_loss`：CTC 分支（快速解码）。
- `decoder_l2r`：左→右注意力解码器。
- `decoder_r2l`：右→左注意力解码器 ← **这就是 U2++ 比 U2 多出来的“++”**。

3.2 运行它

用终端跑（推荐）：

```
python model/u2pp_conformer_min.py
```

用 IDE 按钮跑：

- VSCode: 打开该文件，点右上角 ▷ 三角形 `Run Python File`。

- PyCharm: 打开该文件, 右键编辑区 → Run 'u2pp_conformer_min'。

✅ **检查点:** 终端打印类似下面, 就说明模型结构完全正确 (这是真实运行结果):

```
教学版 U2++ Conformer 前向测试
参数量           : 7.97 M
编码器输入       : (2, 100, 80) (B,T,80)
编码器输出       : (2, 24, 144) (4倍下采样)
CTC 损失         : 8.36
L2R 注意力损失   : 3.93
R2L 注意力损失   : 3.71 ← U2++ 双向解码
联合总损失       : 5.21
结构验证通过     ✅
```

看懂了: 100 帧经过 Conformer 前端下采样 4 倍变成 24 帧; CTC 和左右两个解码器各算了一份损失, 最后合成联合损失。这就是 U2++ 的训练逻辑。

第 4 部分: 生成语音数据集

小白没有录音数据, 我们用微软 TTS (文字转语音) 自动合成"安克耳机指令"音频。
多个发音人 + 多语速 = 自带数据多样性。

4.1 创建 local/generate_anker_corpus.py

新建文件夹 local, 里面建 generate_anker_corpus.py, 内容用桌面标准答案。

你要看懂的地方:

- 顶部 ANKER_COMMANDS: 14 条中文指令 (下一首、打开降噪、接听电话...)。这里就是模型要学识别的目标, 你可以随意增删。
- VOICES: 6 个不同发音人 (男声/女声/口音), 相当于不同说话人。
- 它会把每条指令用每个发音人念一遍, 存成 16kHz 单声道 wav, 并写一份 metadata.tsv (音频路径与文字)。

4.2 运行

```
python local/generate_anker_corpus.py --out data/audio --repeat 2
```

--repeat 2 表示重复 2 轮 (换语速), 数据更多。想更多就调大, 比如 --repeat 4。
这一步需要联网 (要访问微软语音服务)。

✅ **检查点:**

- 终端滚动打印 [1] next_xiaoxiao_r0 -> 下一首 这样的行, 最后显示"完成! 共生成 XXX 条音频"。
- 打开 data/audio 文件夹, 看到一堆 .wav 文件和一个 metadata.tsv。
- 随便双击播放一个 wav, 能听到人声念指令 → 完美。

- ❌ 如果报 未检测到 ffmpeg → 回第 0.3 装 ffmpeg 并重启终端。
- ❌ 如果报网络/超时 → 检查网络, 或重跑 (已生成的会覆盖, 不影响)。

第 5 部分：准备 WeNet 数据格式

WeNet 训练要求特定格式：一份 `data.list`（每行一个 JSON）+ 一份字典 `units.txt`。
这一步把上一步的音频整理成它要的样子，并自动划分 训练/验证/测试 集。

5.1 创建 `local/prepare_data.py`

在 `local` 文件夹里建 `prepare_data.py`，内容用桌面标准答案。

它做了什么：

- 读 `metadata.tsv`，按 8:1:1 划分成 train/dev/test。
- 每个集合写一份 `data.list`，每行形如：
`{"key": "next_Xiaoxiao_r0", "wav": "C:/.../next_Xiaoxiao_r0.wav", "txt": "下一首"}`
- 扫描所有文字里出现的汉字，生成字典 `data/dict/units.txt`，格式 `字 编号`，其中固定包含 `<blank> 0`（CTC 空白符）、`<unk> 1`（未知字）、`<sos/eos>`（句子起止符）。

5.2 运行

```
python local/prepare_data.py --audio_dir data/audio --out_dir data
```

✅ 检查点：

- 终端打印 `train: xxx 条`、`dev: xx 条`、`test: xx 条`、字典大小(含特殊符)：`xx`。
- 出现这些文件：`data/train/data.list`、`data/dev/data.list`、`data/test/data.list`、`data/dict/units.txt`。
- 用 IDE 打开 `data/train/data.list` 看一眼，每行是一个 JSON，路径和文字对得上 → 成功。

第 6 部分：计算 CMVN

CMVN = 对声学特征做“零均值、单位方差”归一化，让训练更稳更快收敛。

6.1 创建 `tools/make_cmvn.py`

新建文件夹 `tools`，里面建 `make_cmvn.py`，内容用桌面标准答案。

它会遍历训练集所有音频，提取 80 维 Fbank 特征，统计均值方差，存成 `global_cmvn`。

6.2 运行

```
python tools/make_cmvn.py --data_list data/train/data.list --out data/train/global_cmvn
```

✅ 检查点：出现进度条，最后打印 `CMVN 完成: xxxxx 帧`，并生成文件 `data/train/global_cmvn` → 成功。

第 7 部分：安装 WeNet 训练代码

前面 `pip` 装的 `wenet` 可能只能推理、不能训练。训练入口 `wenet.bin.train` 在官方仓库里，我们用 `git` 把它下载下来并以“可编辑模式”安装。

7.1 下载并安装

在终端（先 `cd` 回到桌面，让 wenet 和你的项目并排放）：

```
cd C:\Users\ywyuan\Desktop
git clone https://github.com/wenet-e2e/wenet.git
cd wenet
pip install -e .
```

`pip install -e .` 装得有点久，耐心等。装完后记得切回你的项目目录继续后面的步骤：

```
cd C:\Users\ywyuan\Desktop\my-anker-asr
```

7.2 验证

```
python -m wenet.bin.train --help
```

✅ **检查点：**打印出一大段参数帮助（`--config`、`--train_data` 等）→ WeNet 训练代码装好了。

❌ 报 `No module named wenet.bin` → 说明没装成功，回 7.1 重做，注意要在 `wenet` 目录里执行 `pip install -e .`，且 `(.venv)` 必须是激活的。

第 8 部分：写训练配置 + 开始训练

8.1 创建 `conf/train_u2pp_conformer.yaml`

新建文件夹 `conf`，里面建 `train_u2pp_conformer.yaml`，内容用桌面标准答案。

这是整个项目的“大脑配置”，你必须能讲出这几块：

- `encoder: conformer` + `causal: true` + `use_dynamic_chunk: true` → Conformer 编码器，支持流式。
- `decoder: bitransformer` + `r_num_blocks: 3` → 双向解码器，U2++ 的核心。
- `model_conf: ctc_weight: 0.3`（CTC 占比）、`reverse_weight: 0.3`（右→左解码器占比）。
- `dataset_conf: speed_perturb`（速度扰动）、`spec_aug`（频谱掩蔽）两种数据增强。
- `max_epoch: 60`：训练轮数（数据少可以多训几轮）。

8.2 开始训练

复制下面这一整段到终端（注意每行末尾的 ``` 是 PowerShell 的换行符，别漏）：

```
python -m wenet.bin.train \
  --config conf/train_u2pp_conformer.yaml \
  --data_type raw \
  --symbol_table data/dict/units.txt \
  --train_data data/train/data.list \
  --cv_data data/dev/data.list \
  --model_dir exp/u2pp_conformer \
  --cmvn data/train/global_cmvn \
  --num_workers 2 \
  --pin_memory
```

会发生什么：终端开始一轮轮（epoch）打印 loss，数字总体往下降。没显卡用 CPU 也能跑，慢一点，泡杯茶等它。

✅ 检查点：

- 看到类似 Epoch 0 ... loss 5.2 ...、Epoch 1 ... loss 4.1 ...，loss 在下降。
- exp/u2pp_conformer/ 文件夹里陆续出现 0.pt、1.pt ... 以及训练用的 train.yaml。
- 训练跑完（到 max_epoch）会有 final.pt（或用最后一个 *.pt 也行）。

❌ 报"unexpected key / unknown argument"之类 → 是你的 WeNet 版本字段略有不同。

按报错提示，去 conf/train_u2pp_conformer.yaml 删掉/改掉它指出的那一行即可，

核心结构（conformer / bitransformer / ctc）不要动。

❌ 训练太慢 → 把 yaml 里 max_epoch 改小（如 30）、num_blocks 改小（如 4）先跑通流程。

❌ 想中途停 → 终端按 Ctrl + C，已保存的 *.pt 还能用。

第 9 部分：解码评测（算 CER）

用训练好的模型在测试集上识别，并算出 **CER 字错误率**（越低越好）。

9.1 创建 recognize.py

在项目根目录建 recognize.py，内容用桌面标准答案。

它内部调用 WeNet 最稳的解码入口 wenet.bin.recognize，再自动对照标注算 CER。

9.2 运行

```
python recognize.py \
  --config exp/u2pp_conformer/train.yaml \
  --checkpoint exp/u2pp_conformer/final.pt \
  --test_data data/test/data.list \
  --dict data/dict/units.txt \
  --mode attention_rescoring
```

如果没有 final.pt，把 --checkpoint 换成 exp/u2pp_conformer/ 里序号最大的那个 .pt。

✅ **检查点：**终端逐条打印 ✅/❌ 参考：打开降噪 | 识别：打开降噪，最后给出 CER（字错误率）= xx.xx%。

做实验对比（写报告用）：把 --mode 依次换成下面 4 个各跑一次，把 CER 填进 docs/答辩大纲.md 的表格里：

ctc_greedy_search / ctc_prefix_beam_search / attention / attention_rescoring。

通常 attention_rescoring 最准。

💡 数据是合成的、模型也调小了，CER 偏高很正常。报告里如实写"受数据规模/模型规模限制"，重点是**流程完整、方法正确**。想降 CER：第 4 步 `--repeat` 调大造更多数据、yaml 里 `max_epoch` 调大。

第 10 部分：实时识别 Demo

10.1 创建 `infer_demo.py`

项目根目录建 `infer_demo.py`，内容用桌面标准答案。支持"识别一个 wav 文件"或"麦克风现场录音识别"。

10.2 运行

识别测试集里的一个音频（先从 `data/test/data.list` 第一行复制一个 wav 路径）：

```
python infer_demo.py `
--config exp/u2pp_conformer/train.yaml `
--checkpoint exp/u2pp_conformer/final.pt `
--dict data/dict/units.txt `
--wav "data/audio/那个wav的文件名.wav"
```

用麦克风现场说一句（答辩演示神器）：

```
python infer_demo.py `
--config exp/u2pp_conformer/train.yaml `
--checkpoint exp/u2pp_conformer/final.pt `
--dict data/dict/units.txt `
--mic 3
```

运行后会提示"开始录音 3 秒"，你对着麦克风说"打开降噪"，等它识别。

✅ **检查点：**打印 🗣️ 识别结果：【打开降噪】→ 大功告成！

第 11 部分：一键脚本（收尾）

前面你已经一步步手动跑通了。最后把这些命令打包成一键脚本，方便重跑和答辩演示。

在项目根目录建 `run.ps1` (Windows) 和 `run.sh` (如果用 WSL/Linux)，内容用桌面标准答案。以后想从某一步重跑，直接：

```
.\run.ps1 -Stage 3          # 从训练这一步开始
.\run.ps1 -Stage 4 -Stop 4  # 只重跑解码评测
```

全流程回顾（建文件 → 运行 → 验证 的顺序）

步骤	建的文件	运行命令（关键）	验证看到什么
2	requirements.txt	<code>pip install -r requirements.txt</code>	deps ok

步骤	建的文件	运行命令 (关键)	验证看到什么
3	model/u2pp_conformer_min.py	<code>python model/u2pp_conformer_min.py</code>	结构验证通过 
4	local/generate_anker_corpus.py	<code>python local/generate_anker_corpus.py --repeat 2</code>	data/audio 下一堆 wav
5	local/prepare_data.py	<code>python local/prepare_data.py ...</code>	data.list + units.txt
6	tools/make_cmvn.py	<code>python tools/make_cmvn.py ...</code>	global_cmvn
7	(装 WeNet)	<code>python -m wenet.bin.train --help</code>	打印帮助
8	conf/train_u2pp_conformer.yaml	<code>python -m wenet.bin.train ...</code>	loss 下降、出 *.pt
9	recognize.py	<code>python recognize.py ...</code>	CER = xx%
10	infer_demo.py	<code>python infer_demo.py --mic 3</code>	识别结果
11	run.ps1 / run.sh	<code>.\run.ps1</code>	一键全流程

附录：报错急救表

报错关键词	原因	解决
<code>python</code> 不是内部或外部命令	没装 Python 或没勾 PATH	重装 Python, 勾 [Add to PATH]
终端没有 (.venv)	虚拟环境没激活	<code>.\.venv\Scripts\Activate.ps1</code>
禁止运行脚本	PowerShell 策略限制	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
未检测到 ffmpeg	没装/没重开终端	第 0.3 装好后 重开终端
No module named wenet.bin	WeNet 训练代码没装	第 7 部分 <code>git clone</code> + <code>pip install -e .</code>
TTS 网络超时	没联网/被墙	检查网络后重跑第 4 步
训练报 unexpected key	WeNet 版本字段差异	按报错删/改 yaml 对应行
训练巨慢	CPU + 参数偏大	调小 yaml 的 max_epoch / num_blocks
CER 很高	合成数据 + 小模型	正常; 调大 --repeat、max_epoch; 报告里说明
中文输出乱码	Windows GBK 控制台	脚本已加 UTF-8 修复; 若仍乱码可忽略, 不影响功能

 **恭喜你手搓完整 U2++ 语音识别系统!** 原理细节看 docs/原理讲解.md , 报告/PPT/答辩问答看 docs/答辩大纲.md 。祝大作业拿高分! 